

**BỘ KHOA HỌC & CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ I**

---□□□---



**BÁO CÁO MÔN HỌC
THIẾT BỊ NGOẠI VI VÀ KỸ THUẬT GHÉP NỐI**

Đề tài: Chuẩn đóng gói JSON & XML

Giảng viên hướng dẫn : PGS.TS. Trương Cao Dũng

Sinh viên thực hiện : Lê Chí Hiếu – B23DCDT089

Nguyễn Gia Bảo – B23DCDT029

Đào Phú Cường – B23DCDT037

Hoàng Việt Anh – B23DCDT009

Hoàng Thủy Nguyên – B23DCDT180

Hoàng Mỹ Quỳnh – B23DCDT219

Hệ đào tạo đại học : Đại học chính quy

Hà Nội , Tháng 5 năm 2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

This image shows a full page of a document template designed for handwritten notes or answers. It features approximately 20 evenly spaced horizontal dashed lines across the entire width of the page, providing a guide for letter height while allowing for flexibility in writing style. The background is plain white, and there are no margins, headers, or footers visible.

Điểm:..... (bằng chữ:.....)

....., ngày ... tháng ... năm 2026

CÁN BỘ - GIẢNG VIÊN HƯỚNG DẪN

(ký, họ tên)

PGS.TS Trương Cao Dũng

ĐÓNG GÓP CHO BÀI TẬP LỚN

ĐÓNG GÓP CÔNG VIỆC BÀI TẬP LỚN	
Viết code đọc cảm biến DHT11 kết hợp MQTT và publish dữ liệu JSON	Lê Chí Hiếu – B23DCDT089 Đào Phú Cường – B23DCDT089
Xây dựng hệ thống Web hiển thị kết nối	Lê Chí Hiếu – B23DCDT089 Hoàng Thủy Nguyên – B23DCDT180
Xây dựng API lưu trữ vào MongoDB	Lê Chí Hiếu – B23DCDT089 Nguyễn Gia Bảo – B23DCDT029
Xây dựng Server kết nối với API hiển thị lên WEB lưu trữ dữ liệu trong MongoDB	Lê Chí Hiếu – B23DCDT089 Hoàng Việt Anh – B23DCDT009
Kết nối toàn bộ hệ thống và kiểm thử	Lê Chí Hiếu – B23DCDT089 Hoàng Mỹ Quỳnh – B23DCDT219
Viết báo cáo làm slide	Lê Chí Hiếu – B23DCDT089 Hoàng Mỹ Quỳnh – B23DCDT219

Mức độ hoàn thành phân chia công việc

Lê Chí Hiếu – B23DCDT089 – 100% hoàn thành
Hoàng Thủy Nguyên – B23DCDT180 – 100% hoàn thành
Nguyễn Gia Bảo – B23DCDT029 – 100% hoàn thành
Hoàng Việt Anh – B23DCDT009 – 100% hoàn thành
Hoàng Mỹ Quỳnh – B23DCDT219 – 100% hoàn thành
Đào Phú Cường – B23DCDT089 – 100% hoàn thành

LỜI CẢM ƠN

Để hoàn thành tốt đề tài tốt nghiệp này, ngoài sự nỗ lực của bản thân, chúng em còn nhận được rất nhiều sự giúp đỡ quý báu từ các thầy cô cùng các bạn.

Chúng em xin chân thành cảm ơn các thầy cô trong khoa Kỹ thuật Điện tử 1 - Học viện Công nghệ Bru chính Viễn thông đã tận tình giảng dạy và truyền đạt kiến thức cho chúng em trong suốt quá trình học tập tại trường. Những kiến thức và kỹ năng chúng em có được là nhờ sự hướng dẫn nhiệt tình của thầy cô, và đó là nền tảng giúp chúng em hoàn thành đồ án tốt nghiệp này.

Chúng em đặc biệt cảm ơn thầy **Trương Cao Dũng** đã dành thời gian chỉ bảo, hỗ trợ và đưa ra những góp ý quý báu trong quá trình thực hiện đề tài " Chuẩn đóng gói JSON & XML ". Những lời khuyên của thầy đã giúp em có định hướng rõ ràng và khắc phục các khó khăn gặp phải.

Chúng em xin trân trọng cảm ơn tất cả những sự giúp đỡ quý báu đó!

Hà Nội, tháng 5 năm 2026

TÓM TẮT

Trong bối cảnh công nghệ ngày càng phát triển mạnh mẽ, các hệ thống nhúng không còn hoạt động đơn lẻ mà ngày càng gắn kết chặt chẽ với nhau thông qua mạng và các nền tảng số. Điều này đặt ra một yêu cầu quan trọng: dữ liệu phải được tổ chức, truyền tải và xử lý một cách thống nhất, rõ ràng và hiệu quả. Chính vì vậy, việc chuẩn hóa dữ liệu bằng các định dạng như JSON và XML không chỉ là một lựa chọn, mà gần như đã trở thành nền tảng cốt lõi trong các hệ thống hiện đại. Đặc biệt, trong các bài toán liên quan đến IoT và kỹ thuật ghép nối thiết bị, vai trò của việc đóng gói dữ liệu càng trở nên quan trọng hơn bao giờ hết.

Khi xây dựng một hệ thống giám sát và điều khiển sử dụng vi điều khiển như ESP32 kết hợp với giao thức MQTT, dữ liệu không còn chỉ nằm ở một nơi duy nhất. Nó liên tục di chuyển: từ cảm biến, qua mạng, đến server, rồi hiển thị lên giao diện người dùng. Nếu không có một chuẩn chung, dữ liệu sẽ trở nên rời rạc, khó hiểu và rất dễ gây lỗi trong quá trình xử lý. Việc sử dụng JSON giúp dữ liệu trở nên gọn gàng, dễ đọc, dễ phân tích và đặc biệt phù hợp với các ứng dụng web hiện đại. Trong khi đó, XML mang lại tính chặt chẽ, có cấu trúc rõ ràng, phù hợp với những hệ thống yêu cầu kiểm soát dữ liệu nghiêm ngặt. Sự tồn tại song song của hai chuẩn này cho phép hệ thống linh hoạt hơn trong nhiều tình huống khác nhau.

Không chỉ dừng lại ở việc truyền dữ liệu, chuẩn hóa JSON/XML còn đóng vai trò như một “ngôn ngữ chung” giúp các thành phần trong hệ thống hiểu được nhau. Từ thiết bị nhúng gửi dữ liệu cảm biến, đến server xử lý và lưu trữ vào cơ sở dữ liệu, rồi đến API cung cấp dữ liệu cho web – tất cả đều cần một định dạng thống nhất. Nhờ đó, việc phát triển hệ thống trở nên dễ dàng hơn, giảm thiểu sai sót và tiết kiệm thời gian trong quá trình debug. Đặc biệt, khi hệ thống mở rộng (thêm thiết bị, thêm tính năng), việc đã có sẵn một chuẩn dữ liệu sẽ giúp tích hợp nhanh chóng mà không cần thay đổi toàn bộ kiến trúc.

Trong thực tế triển khai, việc chuẩn hóa dữ liệu còn mang lại nhiều lợi ích rõ rệt. Ví dụ, trong hệ thống giám sát nhiệt độ và độ ẩm, dữ liệu từ ESP32 được đóng gói thành JSON và gửi qua MQTT. Server nhận dữ liệu, kiểm tra tính hợp lệ, lưu vào MongoDB, sau đó cung cấp lại qua API để hiển thị lên web. Nhờ có cấu trúc dữ liệu rõ ràng, toàn bộ quá trình này diễn ra trơn tru, từ việc hiển thị biểu đồ, lưu lịch sử, cho đến xuất file CSV phục vụ phân tích. Người dùng không cần quan tâm dữ liệu đi qua bao nhiêu bước, mà chỉ thấy được kết quả cuối cùng một cách trực quan và dễ hiểu.

Bên cạnh những lợi ích, việc chuẩn hóa dữ liệu cũng đặt ra một số thách thức nhất định. Người phát triển cần phải thiết kế schema hợp lý ngay từ đầu, đảm bảo vừa đủ thông tin nhưng không dư thừa. Ngoài ra, việc xử lý dữ liệu sai định dạng, thiếu trường hoặc lỗi kiểu dữ liệu cũng cần được kiểm soát chặt chẽ để tránh ảnh hưởng đến toàn bộ hệ thống. Điều này đòi hỏi sự cẩn thận và tư duy hệ thống rõ ràng trong quá trình thiết kế.

Tóm lại, trong môn học Thiết bị ngoại vi và kỹ thuật ghép nối, việc xây dựng hệ thống với chuẩn đóng gói JSON và XML không chỉ giúp sinh viên hiểu rõ cách các thiết bị giao tiếp với nhau, mà còn tiếp cận gần hơn với các hệ thống thực tế đang được sử dụng trong công nghiệp. Đây không đơn thuần là một bài tập kỹ thuật, mà còn là bước đệm quan trọng giúp hình thành tư duy thiết kế hệ thống, nơi mà dữ liệu đóng vai trò trung tâm. Với nền tảng này, việc phát triển các ứng dụng IoT phức tạp hơn trong tương lai sẽ trở nên dễ dàng và hiệu quả hơn rất nhiều.

LỜI CAM ĐOAN

Chúng em xin cam đoan nội dung trình bày trong báo cáo “ Chuẩn đóng gói JSON/XML ” này là quá trình nghiên cứu và tìm hiểu của chúng em dưới sự hướng dẫn của thầy Trương Cao Dũng . Những nội dung nghiên cứu và kết quả trong đồ án là hoàn toàn trung thực, có trích dẫn đầy đủ. Nếu như không đúng như đã nêu trên, chúng em xin chịu hoàn toàn trách nhiệm về đề tài của mình.

Hà Nội, tháng 5 năm 2026

Đại diện trưởng nhóm

Sinh viên

Lê Chí Hiếu

MỤC LỤC

LỜI CẢM ƠN.....	3
TÓM TẮT	5
LỜI CAM ĐOAN	7
MỤC LỤC	8
LÝ DO CHỌN ĐỀ TÀI.....	9
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI	10
CHƯƠNG 2: XÂY DỰNG HỆ THỐNG	13
CHƯƠNG 3: TRIỂN KHAI HỆ THỐNG	25
CHƯƠNG 4: THỬ NGHIỆM VÀ ĐÁNH GIÁ	29
CHƯƠNG 5: KẾT LUẬN VÀ KIẾN NGHỊ.....	31
TÀI LIỆU THAM KHẢO	32

LÝ DO CHỌN ĐỀ TÀI

Hệ thống chuẩn hóa dữ liệu trong các ứng dụng IoT đang trở thành một yếu tố quan trọng nhờ nhu cầu kết nối và trao đổi dữ liệu giữa các thiết bị ngày càng gia tăng. Trong các hệ thống giám sát và điều khiển, dữ liệu từ cảm biến cần được truyền đi, xử lý và hiển thị một cách chính xác và nhất quán. Việc sử dụng các định dạng như JSON và XML giúp chuẩn hóa dữ liệu, từ đó đảm bảo các thành phần trong hệ thống có thể “hiểu” và tương tác với nhau một cách hiệu quả. Đặc biệt, trong các hệ thống nhà thông minh hoặc giám sát môi trường, việc chuẩn hóa dữ liệu giúp giảm thiểu lỗi, tăng độ ổn định và dễ dàng mở rộng trong tương lai.

JSON là định dạng dữ liệu nhẹ, dễ đọc và dễ tích hợp với các ứng dụng web hiện đại, rất phù hợp với các hệ thống sử dụng MQTT và API. Trong khi đó, XML cung cấp cấu trúc dữ liệu chặt chẽ hơn, phù hợp với các hệ thống yêu cầu tính kiểm soát cao. Việc kết hợp và áp dụng hai chuẩn này giúp người học hiểu rõ hơn về cách đóng gói, truyền và xử lý dữ liệu trong thực tế. Ngoài ra, việc triển khai chuẩn hóa dữ liệu trên các thiết bị nhúng như ESP32 còn giúp tối ưu tài nguyên, đảm bảo hệ thống hoạt động hiệu quả ngay cả với phần cứng hạn chế, đồng thời tạo nền tảng cho việc phát triển các ứng dụng IoT phức tạp hơn trong tương lai.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Tổng quan về hệ thống

Hệ thống giám sát và điều khiển thiết bị sử dụng chuẩn đóng gói dữ liệu JSON/XML là một ứng dụng tiêu biểu trong lĩnh vực IoT, cho phép thu thập, xử lý và truyền dữ liệu giữa các thiết bị nhúng và hệ thống trung tâm một cách hiệu quả. Đề án này tập trung xây dựng một hệ thống đơn giản có khả năng giám sát các thông số môi trường như nhiệt độ và độ ẩm, đồng thời cho phép điều khiển thiết bị (LED) thông qua giao thức MQTT. Mục tiêu chính của hệ thống là đảm bảo dữ liệu được chuẩn hóa, truyền tải chính xác và hiển thị trực quan trên giao diện web, từ đó giúp người dùng dễ dàng theo dõi và điều khiển thiết bị từ xa.

Hệ thống sử dụng vi điều khiển ESP32 làm thiết bị thu thập dữ liệu từ cảm biến và gửi lên server thông qua giao thức MQTT. Dữ liệu sau khi được thu thập sẽ được đóng gói theo định dạng JSON nhằm đảm bảo tính thống nhất và dễ dàng xử lý. Tại phía server, dữ liệu được tiếp nhận, kiểm tra tính hợp lệ và lưu trữ vào cơ sở dữ liệu MongoDB. Đồng thời, hệ thống cũng cung cấp API để truyền dữ liệu đến giao diện web, giúp hiển thị thông tin theo thời gian thực. Ngoài ra, XML cũng có thể được sử dụng như một định dạng thay thế nhằm giúp người học hiểu rõ hơn về các phương pháp chuẩn hóa dữ liệu khác nhau trong thực tế.

Trong hệ thống này, dữ liệu đóng vai trò trung tâm và được luân chuyển liên tục giữa các thành phần. Từ cảm biến gửi dữ liệu, đến server xử lý, rồi hiển thị lên giao diện người dùng – tất cả đều cần một định dạng chung để đảm bảo sự đồng bộ. Việc sử dụng JSON giúp giảm độ phức tạp trong truyền tải và xử lý dữ liệu, trong khi XML cung cấp một cấu trúc rõ ràng và chặt chẽ hơn khi cần thiết. Nhờ đó, hệ thống có thể hoạt động ổn định, dễ mở rộng và phù hợp với nhiều ứng dụng IoT khác nhau.

Mô hình hoạt động của hệ thống:

1. Thu thập dữ liệu : ESP32 đọc dữ liệu từ cảm biến (nhiệt độ, độ ẩm) và trạng thái thiết bị
2. Đóng gói dữ liệu : Dữ liệu được chuẩn hóa và đóng gói theo định dạng JSON (hoặc XML) trước khi truyền đi
3. Truyền dữ liệu qua MQTT (Message Queuing Telemetry Transport): Dữ liệu được gửi đến broker MQTT, sau đó server sẽ subscribe để nhận dữ liệu.
4. Xử lý và lưu trữ : Server kiểm tra dữ liệu, loại bỏ dữ liệu không hợp lệ và lưu vào MongoDB.
5. Cung cấp API và hiển thị : API cung cấp dữ liệu cho web, giúp hiển thị biểu đồ, trạng thái thiết bị và nhật ký hoạt động theo thời gian thực.
6. Điều khiển thiết bị : Người dùng gửi lệnh từ web → server → MQTT → ESP32 để điều khiển bật/tắt thiết bị.

1.2. Công nghệ sử dụng

Trong đồ án này, các công nghệ phổ biến và thực tiễn trong lĩnh vực IoT và phát triển hệ thống nhúng đã được sử dụng để xây dựng một hệ thống giám sát và điều khiển thiết bị hoàn chỉnh. Các công nghệ chủ yếu bao gồm:

- ESP32(ESP32-C3-SUPERMINI): ESP32 là vi điều khiển mạnh mẽ, tích hợp WiFi và Bluetooth, rất phù hợp cho các ứng dụng IoT. Trong hệ thống này, ESP32 được sử dụng để đọc dữ liệu từ cảm biến (nhiệt độ, độ ẩm) và gửi dữ liệu lên server thông qua giao thức MQTT. Ngoài ra, ESP32 còn nhận lệnh điều khiển từ server để bật/tắt các thiết bị như LED, giúp hệ thống hoạt động hai chiều một cách linh hoạt.

- MQTT (Message Queuing Telemetry Transport): MQTT là giao thức truyền thông nhẹ, được thiết kế tối ưu cho các thiết bị có tài nguyên hạn chế. Trong hệ thống, MQTT đóng vai trò trung gian giúp truyền dữ liệu giữa ESP32 và server. Nhờ cơ chế publish/subscribe, dữ liệu được truyền tải nhanh chóng, giảm độ trễ và đảm bảo tính hiệu quả trong giao tiếp giữa các thành phần.

- JSON (JavaScript Object Notation): JSON là định dạng dữ liệu nhẹ, dễ đọc và dễ xử lý, được sử dụng để đóng gói dữ liệu cảm biến trước khi gửi qua MQTT. Việc sử dụng JSON giúp chuẩn hóa dữ liệu, đảm bảo tính nhất quán giữa các thành phần trong hệ thống, đồng thời thuận tiện cho việc xử lý ở phía server và hiển thị trên web.

- XML (eXtensible Markup Language): XML là định dạng dữ liệu có cấu trúc chặt chẽ, thường được sử dụng trong các hệ thống yêu cầu kiểm soát dữ liệu cao. Trong đồ án này, XML được nghiên cứu như một phương pháp chuẩn hóa dữ liệu bổ sung, giúp người học hiểu rõ hơn về các cách tổ chức và trao đổi dữ liệu trong các hệ thống khác nhau.

- Node.js (Server Backend): Node.js được sử dụng để xây dựng server nhận dữ liệu từ MQTT, xử lý và lưu trữ vào cơ sở dữ liệu. Ngoài ra, server còn đảm nhiệm việc cung cấp API để web có thể truy cập và hiển thị dữ liệu theo thời gian thực. Nhờ khả năng xử lý bất đồng bộ, Node.js giúp hệ thống hoạt động mượt mà và hiệu quả.

- MongoDB : MongoDB là cơ sở dữ liệu NoSQL, phù hợp với dữ liệu dạng JSON. Trong hệ thống, MongoDB được sử dụng để lưu trữ dữ liệu cảm biến theo thời gian, giúp dễ dàng truy vấn và phân tích dữ liệu sau này.

- Express.js (API): Express.js là framework của Node.js, được sử dụng để xây dựng các API cung cấp dữ liệu cho giao diện web. API cho phép lấy dữ liệu mới nhất, phục vụ cho việc hiển thị biểu đồ và trạng thái hệ thống.

- Giao diện Web (HTML, TailwindCSS, Chart.js): Giao diện web được xây dựng bằng HTML kết hợp với TailwindCSS để tạo bố cục đẹp và dễ sử dụng. Chart.js được sử dụng để hiển thị dữ liệu dưới dạng biểu đồ trực quan theo thời gian thực, giúp người dùng dễ dàng theo dõi sự thay đổi của nhiệt độ và độ ẩm.

1.3. Mục tiêu của nhóm

Mục tiêu của nhóm là xây dựng một hệ thống giám sát và điều khiển thiết bị đơn giản nhưng hiệu quả, có khả năng thu thập, chuẩn hóa và truyền dữ liệu giữa các thành phần trong hệ thống thông qua định dạng JSON/XML. Hệ thống cho phép theo dõi các thông số môi trường như nhiệt độ, độ ẩm và điều khiển thiết bị (LED) từ xa thông qua giao diện web. Mục tiêu cụ thể của nhóm bao gồm:

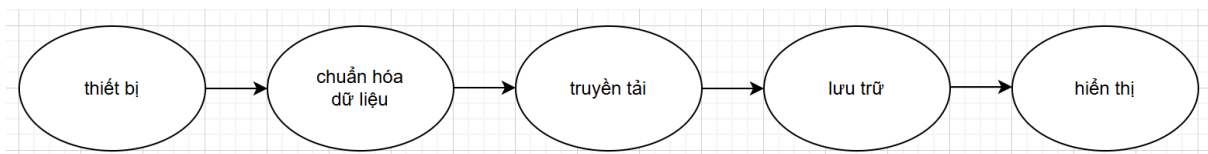
1. Chuẩn hóa dữ liệu bằng JSON/XML: Xây dựng cấu trúc dữ liệu thống nhất cho toàn bộ hệ thống, đảm bảo dữ liệu từ ESP32 gửi lên luôn đúng định dạng (JSON/XML), đầy đủ các trường cần thiết như nhiệt độ, độ ẩm, trạng thái thiết bị và thời gian.
2. Thu thập và truyền dữ liệu qua MQTT: Thiết kế hệ thống để ESP32 có thể đọc dữ liệu từ cảm biến và truyền lên server thông qua giao thức MQTT một cách ổn định, liên tục và có độ trễ thấp.
3. Xử lý và lưu trữ dữ liệu: Xây dựng server sử dụng Node.js để nhận dữ liệu từ MQTT, kiểm tra tính hợp lệ, loại bỏ dữ liệu lỗi và lưu trữ vào cơ sở dữ liệu MongoDB phục vụ cho việc truy vấn và phân tích sau này.
4. Xây dựng API cung cấp dữ liệu: Phát triển các API bằng Express.js để cung cấp dữ liệu cho giao diện web, bao gồm việc lấy dữ liệu mới nhất và hỗ trợ hiển thị theo thời gian thực.
5. Hiển thị dữ liệu trực quan trên web: Xây dựng giao diện web hiển thị thông tin nhiệt độ, độ ẩm, trạng thái thiết bị và biểu đồ dữ liệu theo thời gian, giúp người dùng dễ dàng theo dõi hệ thống.
6. Điều khiển thiết bị từ xa: Cho phép người dùng gửi lệnh điều khiển từ giao diện web thông qua MQTT để bật/tắt thiết bị, đồng thời đảm bảo hệ thống phản hồi nhanh và chính xác.

Với các mục tiêu trên, nhóm hướng đến việc xây dựng một hệ thống IoT hoàn chỉnh, trong đó dữ liệu được chuẩn hóa, xử lý và truyền tải một cách hiệu quả. Hệ thống không chỉ giúp người dùng giám sát và điều khiển thiết bị một cách thuận tiện, mà còn tạo nền tảng cho việc phát triển các ứng dụng IoT nâng cao trong tương lai.

CHƯƠNG 2: XÂY DỰNG HỆ THỐNG

2.1. Kiến trúc model

Trong đề tài “chuẩn đóng gói dữ liệu JSON/XML cho hệ thống IoT”, kiến trúc hệ thống không còn là xử lý tín hiệu âm thanh như TinyML, mà chuyển sang xử lý dữ liệu cảm biến và truyền thông giữa các thành phần. Tuy nhiên, về bản chất, luồng xử lý vẫn tuân theo một pipeline rõ ràng:



Cụ thể, hệ thống được xây dựng với các bước chính như sau:

2.1.1. Thu thập dữ liệu (Data Acquisition)

Dữ liệu được thu thập từ cảm biến (DHT11) thông qua vi điều khiển ESP32-C3. Các thông số bao gồm:

- Nhiệt độ (temperature)
- Độ ẩm (humidity)
- Trạng thái LED
- Chế độ hoạt động (auto/manual)

Khác với tín hiệu analog trong xử lý âm thanh, dữ liệu ở đây đã ở dạng số, nhưng vẫn tồn tại vấn đề không đồng nhất định dạng giữa các thiết bị.

Ví dụ:

- ESP gửi dạng thô
 - MQTT truyền dạng string
 - Backend cần object chuẩn
- Vì vậy cần bước chuẩn hóa.

Dưới đây là đoạn code đọc cảm biến DHT11 đã kết nối với MQTT và gửi dữ liệu định dạng chuẩn JSON :

```
#include <ArduinoJson.h>
#include <DHT.h>
#include <PubSubClient.h>
#include <WiFi.h>
```

```
// ===== WIFI =====  
const char *ssid = "TEN_WIFI";  
const char *password = "MAT_KHAU_WIFI";  
  
// ===== MQTT =====  
const char *mqtt_server = "broker.emqx.io";  
const int mqtt_port = 1883;  
const char *topic_pub = "iot/esp32c3_01/data";  
const char *topic_ctrl = "esp32/dht11/control";  
const char *topic_status = "esp32/dht11/status";  
  
// ===== DHT11 =====  
#define DHTPIN 2  
#define DHTTYPE DHT11  
DHT dht(DHTPIN, DHTTYPE);  
  
// ===== LED =====  
#define LED_TEMP 9  
#define LED_HUM 0  
  
// ===== BIẾN TOÀN CỤC =====  
WiFiClient espClient;  
PubSubClient client(espClient);  
bool autoMode = true;  
bool manualLedTemp = false;  
bool manualLedHum = false;  
float currentTemp = 0;  
float currentHum = 0;  
unsigned long lastPublish = 0;
```

```
const long publishInterval = 2000;

// ===== CONNECT WIFI =====

void setup_wifi() {
    delay(1000);
    Serial.println("\nConnecting to WiFi...");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi connected!");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());
}

// ===== MQTT CALLBACK =====

void callback(char *topic, byte *payload, unsigned int length) {
    String message;
    for (unsigned int i = 0; i < length; i++) {
        message += (char)payload[i];
    }

    Serial.print("MQTT received: ");
    Serial.println(message);

    if (String(topic) == topic_ctrl) {
        StaticJsonDocument<128> doc;
        DeserializationError error = deserializeJson(doc, message);
        if (error)
```

```

return;

const char *command = doc["command"];
const char *value = doc["value"];

if (strcmp(command, "mode") == 0) {
    if (strcmp(value, "auto") == 0) {
        autoMode = true;
        Serial.println("Auto mode ON");
        updateLEDsBySensor();
    } else if (strcmp(value, "manual") == 0) {
        autoMode = false;
        Serial.println("Manual mode ON");
    }
} else if (strcmp(command, "led") == 0) {
    if (!autoMode) {
        if (strcmp(value, "temp") == 0) {
            manualLedTemp = !manualLedTemp;
            digitalWrite(LED_TEMP, manualLedTemp);
        } else if (strcmp(value, "hum") == 0) {
            manualLedHum = !manualLedHum;
            digitalWrite(LED_HUM, manualLedHum);
        }
        publishStatus();
    }
}
}

// ===== UPDATE LED AUTO =====

```

```
void updateLEDsBySensor() {
  if (autoMode) {
    bool newLedTemp = (currentTemp > 30.0);
    bool newLedHum = (currentHum > 70.0);

    if (newLedTemp != manualLedTemp) {
      manualLedTemp = newLedTemp;
      digitalWrite(LED_TEMP, manualLedTemp);
    }
    if (newLedHum != manualLedHum) {
      manualLedHum = newLedHum;
      digitalWrite(LED_HUM, manualLedHum);
    }
  }
}
```

```
// ===== PUBLISH STATUS =====
```

```
void publishStatus() {
  StaticJsonDocument<128> doc;
  doc["led_hum"] = manualLedHum;
  doc["led_temp"] = manualLedTemp;
  doc["mode"] = autoMode ? "auto" : "manual";
  doc["timestamp"] = millis() / 1000;

  char buffer[128];
  serializeJson(doc, buffer);
  client.publish(topic_status, buffer);
}
```

```
// ===== RECONNECT MQTT =====
```

```

void reconnect() {
  while (!client.connected()) {
    Serial.print("Connecting MQTT...");
    String clientId = "esp32c3_" + String(random(0xffff), HEX);
    if (client.connect(clientId.c_str())) {
      Serial.println("OK");
      client.subscribe(topic_ctrl);
      publishStatus();
    } else {
      Serial.print("Fail, rc=");
      Serial.print(client.state());
      Serial.println(" retry...");
      delay(2000);
    }
  }
}

// ===== PUBLISH SENSOR =====
void publishData(float temp, float hum) {
  StaticJsonDocument<256> doc;

  // Thông tin cơ bản
  doc["device_id"] = "esp32_c3_super_mini";
  doc["timestamp"] = millis() / 1000;

  // Dữ liệu cảm biến
  doc["temperature"] = temp;
  doc["humidity"] = hum;

  // ===== TRẠNG THÁI LED =====

```

```
doc["led_temp_state"] = manualLedTemp; // true = BẬT, false = TẮT
doc["led_hum_state"] = manualLedHum; // true = BẬT, false = TẮT

// Chế độ hiện tại
doc["mode"] = autoMode ? "auto" : "manual";

char buffer[256];
serializeJson(doc, buffer);
client.publish(topic_pub, buffer);
Serial.print("Publish: ");
Serial.println(buffer);
}

// ===== SETUP =====
void setup() {
  Serial.begin(115200);
  pinMode(LED_TEMP, OUTPUT);
  pinMode(LED_HUM, OUTPUT);
  digitalWrite(LED_TEMP, LOW);
  digitalWrite(LED_HUM, LOW);
  dht.begin();
  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(callback);
}

// ===== LOOP =====
void loop() {
  if (!client.connected()) {
    reconnect();
  }
}
```

```
}  
client.loop();  
  
if (millis() - lastPublish >= publishInterval) {  
    lastPublish = millis();  
  
    float temp = dht.readTemperature();  
    float hum = dht.readHumidity();  
  
    if (isnan(temp) || isnan(hum)) {  
        Serial.println("DHT11 read failed!");  
        return;  
    }  
  
    currentTemp = temp;  
    currentHum = hum;  
  
    Serial.print("Temp: ");  
    Serial.print(temp);  
    Serial.print(" C | Hum: ");  
    Serial.print(hum);  
    Serial.println(" %");  
    Serial.print("LED Temp: ");  
    Serial.print(manualLedTemp ? "ON" : "OFF");  
    Serial.print(" | LED Hum: ");  
    Serial.println(manualLedHum ? "ON" : "OFF");  
  
    if (autoMode) {  
        updateLEDsBySensor();  
    }  
}
```

```
publishData(temp, hum);  
publishStatus();  
}  
delay(100);  
}
```

Kết quả thu được sẽ như sau :

```
15:07:10.356 -> LED Temp: OFF | LED Hum: ON  
15:07:10.356 -> Publish:  
{"device_id":"esp32_c3_super_mini","timestamp":68,"temperature":27.7,"humidity":72,"led_temp_state":false,"led_hum_state":true,"mode":"auto"}  
15:07:12.400 -> Temp: 27.70 C | Hum: 72.00 %  
15:07:12.400 -> LED Temp: OFF | LED Hum: ON  
15:07:12.400 -> Publish:  
{"device_id":"esp32_c3_super_mini","timestamp":70,"temperature":27.7,"humidity":72,"led_temp_state":false,"led_hum_state":true,"mode":"auto"}  
15:07:14.405 -> Temp: 27.70 C | Hum: 72.00 %  
15:07:14.405 -> LED Temp: OFF | LED Hum: ON  
15:07:14.405 -> Publish:  
{"device_id":"esp32_c3_super_mini","timestamp":72,"temperature":27.7,"humidity":72,"led_temp_state":false,"led_hum_state":true,"mode":"auto"}  
15:07:16.429 -> Temp: 27.70 C | Hum: 72.00 %  
15:07:16.429 -> LED Temp: OFF | LED Hum: ON  
15:07:16.471 -> Publish:  
{"device_id":"esp32_c3_super_mini","timestamp":74,"temperature":27.7,"humidity":72,"led_temp_state":false,"led_hum_state":true,"mode":"auto"}  
15:07:18.496 -> Temp: 27.70 C | Hum: 71.00 %  
15:07:18.496 -> LED Temp: OFF | LED Hum: ON  
15:07:18.496 -> Publish:  
{"device_id":"esp32_c3_super_mini","timestamp":76,"temperature":27.7,"humidity":71,"led_temp_state":false,"led_hum_state":true,"mode":"auto"}  
LOLIN C3 Mini on COM3
```

2.1.2. Chuẩn hóa dữ liệu (Data Normalization)

Đây là trái tim của đề tài.

Thay vì để mỗi thiết bị gửi dữ liệu theo cách riêng, hệ thống định nghĩa một schema JSON chuẩn:

```
JSON
{
  "device_id": "esp32_c3",
  "temperature": 28.5,
  "humidity": 80,
  "timestamp": 123456,
  "led_temp_state": true,
  "led_hum_state": false,
  "mode": "auto"
}
```

Việc chuẩn hóa này giúp:

- Dữ liệu đồng nhất
- Dễ lưu database
- Dễ parse frontend
- Dễ mở rộng sau này

Nếu dùng XML, cấu trúc tương đương:

```
XML
<device>
  <id>esp32_c3</id>
  <temperature>28.5</temperature>
  <humidity>80</humidity>
</device>
```

2.1.3. Truyền dữ liệu qua MQTT

Sau khi chuẩn hóa, dữ liệu được gửi qua giao thức MQTT.

Luồng hoạt động:

1. ESP32 publish dữ liệu lên broker
2. Server (Node.js) subscribe
3. Nhận dữ liệu → xử lý → lưu DB

MQTT đóng vai trò:

- Nhẹ
- Real-time
- Phù hợp IoT

2.1.4. Xử lý và lưu trữ dữ liệu

Backend sử dụng Node.js để:

- Nhận dữ liệu MQTT
- Parse JSON
- Lưu vào MongoDB

Dữ liệu trong database có dạng:

```
JSON
{
  "_id": "...",
  "temperature": 28.9,
  "humidity": 85,
  "createdAt": "2026-04-28T03:45:45.390Z"
}
```

Tại đây xuất hiện vấn đề thực tế: Timezone (UTC vs Việt Nam) cần xử lý để hiển thị

2.1.5. Cung cấp API cho frontend

Server tạo API: GET /api/latest

Frontend sẽ gọi API này để lấy dữ liệu mới nhất.

Vai trò API:

- Cầu nối giữa database và web

-
- Chuẩn hóa đầu ra
 - Tách biệt hệ thống

2.1.6. Hiển thị và trực quan hóa dữ liệu

Frontend (HTML + JS) thực hiện:

- Hiển thị nhiệt độ, độ ẩm
- Vẽ biểu đồ bằng Chart.js
- Hiển thị log
- Điều khiển LED qua MQTT

Đây là bước cuối: biến dữ liệu → thông tin → trải nghiệm người dùng

2.2. Luồng hoạt động tổng thể của hệ thống

Hệ thống hoạt động theo chu trình liên tục:

1. ESP32 đọc dữ liệu cảm biến
2. Đóng gói JSON
3. Gửi qua MQTT
4. Server nhận và lưu MongoDB
5. API cung cấp dữ liệu
6. Web fetch và hiển thị

Đây chính là mô hình **IoT hoàn chỉnh end-to-end**

2.3. Ý nghĩa của chuẩn hóa JSON/XML trong hệ thống

Điểm quan trọng nhất của đề tài không nằm ở việc “hiển thị web”, mà nằm ở: **chuẩn hóa dữ liệu**

Nếu không có chuẩn:

- ESP gửi kiểu A
- Server hiểu kiểu B
- Web cần kiểu C

→ hệ thống sẽ **vỡ ngay**

Chuẩn JSON/XML giúp:

- Đồng bộ toàn hệ thống
- Dễ debug
- Dễ mở rộng (thêm sensor, thêm thiết bị)

CHƯƠNG 3: TRIỂN KHAI HỆ THỐNG

3.1. Xây dựng bộ xử lý logic dữ liệu và điều khiển hệ thống

Trong hệ thống này, bộ xử lý logic đóng vai trò trung tâm, chịu trách nhiệm tiếp nhận dữ liệu từ thiết bị IoT, xử lý, chuẩn hóa và đưa ra các hành động điều khiển phù hợp. Khác với hệ thống nhận diện giọng nói, nơi tín hiệu âm thanh là đầu vào, ở đây dữ liệu đầu vào là các thông số cảm biến như nhiệt độ, độ ẩm và trạng thái thiết bị.

Quá trình xử lý logic được thực hiện theo các bước sau:

1. Thu thập và tiếp nhận dữ liệu

Dữ liệu được gửi từ vi điều khiển ESP32-C3 thông qua giao thức MQTT. Mỗi gói tin được đóng gói dưới dạng JSON và chứa đầy đủ thông tin về trạng thái hệ thống.

Ở phía server, quá trình này được xử lý thông qua việc subscribe vào các topic MQTT, ví dụ:

Javascript

```
client.on("message", (topic, message) => {  
  const data = JSON.parse(message.toString());  
});
```

2. Chuẩn hóa và xử lý dữ liệu

Sau khi tiếp nhận, dữ liệu sẽ được chuẩn hóa theo một cấu trúc thống nhất. Đây là bước quan trọng nhằm đảm bảo toàn bộ hệ thống (ESP32, Server, Web) đều hiểu dữ liệu theo cùng một cách.

Ví dụ một bản ghi chuẩn:

JSON

```
{  
  "device_id": "esp32_c3",  
  "temperature": 28.7,  
  "humidity": 85,  
  "led_temp_state": true,  
  "led_hum_state": false,  
  "mode": "auto"  
}
```

Việc chuẩn hóa giúp:

- Tránh sai lệch dữ liệu
- Dễ dàng lưu trữ
- Dễ dàng hiển thị và phân tích

3. Lưu trữ dữ liệu vào cơ sở dữ liệu

Sau khi xử lý, dữ liệu được lưu vào MongoDB để phục vụ cho việc truy xuất và hiển thị sau này.

Trong code, quá trình này được thực hiện thông qua các lệnh như:

```
await SensorData.create(data);
```

MongoDB sẽ tự động sinh thêm các trường như:

- createdAt
- updatedAt

Điều này giúp hệ thống theo dõi được lịch sử dữ liệu theo thời gian thực.

4. Điều khiển thiết bị thông qua MQTT

Ngoài việc nhận dữ liệu, hệ thống còn cho phép gửi lệnh điều khiển ngược lại thiết bị ESP32.

Frontend sẽ gửi lệnh qua MQTT dưới dạng:

```
JSON
{
  "command": "led",
  "value": "temp"
}
```

Server hoặc trực tiếp Web sẽ publish lên broker:

```
client.publish("esp32/dht11/control", payload);
```

ESP32 nhận lệnh và thực hiện:

- Bật/tắt LED
- Chuyển chế độ auto/manual

3.2. Thiết lập backend và API

Sau khi xây dựng logic xử lý dữ liệu, bước tiếp theo là thiết lập backend để cung cấp API cho frontend.

1. Xây dựng server Node.js

Backend được xây dựng bằng Node.js kết hợp Express, cho phép tạo các API đơn giản nhưng hiệu quả.

Ví dụ API lấy dữ liệu mới nhất:

```
app.get("/api/latest", async (req, res) => {  
  const data = await SensorData.findOne().sort({ createdAt: -1 });  
  res.json(data);  
});
```

API này đóng vai trò:

- Cung cấp dữ liệu cho web
- Tách biệt frontend và database

2. Xử lý vấn đề CORS

Trong quá trình phát triển, một vấn đề phổ biến là lỗi CORS khi frontend và backend chạy khác port.

Giải pháp là sử dụng middleware:

```
const cors = require("cors");  
app.use(cors());
```

Điều này cho phép web có thể gọi API một cách hợp lệ.

3. Đồng bộ thời gian (Timezone)

Dữ liệu trong MongoDB mặc định lưu theo UTC. Để hiển thị đúng giờ Việt Nam, hệ thống cần chuyển đổi timezone.

Ví dụ:

```
new Date(data.createdAt).toLocaleString("vi-VN")
```

Điều này giúp người dùng dễ dàng theo dõi dữ liệu theo thời gian thực tế.

3.3. Xây dựng giao diện web và tích hợp hệ thống

Frontend đóng vai trò là nơi tương tác trực tiếp với người dùng, hiển thị dữ liệu và gửi lệnh điều khiển.

1. Hiển thị dữ liệu cảm biến

Dữ liệu được lấy từ API và hiển thị lên giao diện:

```
document.getElementById("temp").innerHTML = currentTemp + "°C";
```

Các thông tin hiển thị gồm:

- Nhiệt độ
- Độ ẩm
- Trạng thái thiết bị

2. Vẽ biểu đồ trực quan

Hệ thống sử dụng thư viện Chart.js để hiển thị dữ liệu theo thời gian.

```
new Chart(ctx, {  
  type: 'line',  
  data: {...}  
});
```

Biểu đồ giúp:

- Theo dõi xu hướng
- Phát hiện bất thường

3. Gửi lệnh điều khiển thiết bị

Người dùng có thể điều khiển hệ thống thông qua các nút bấm:

```
sendMQTT("led", "temp");
```

Các chức năng bao gồm:

- Bật/tắt LED
- Chuyển chế độ auto/manual

4. Xuất dữ liệu CSV

Hệ thống hỗ trợ xuất dữ liệu để lưu trữ hoặc phân tích:

```
let csv = logs.map(...).join("\n");
```

Tính năng này giúp:

- Lưu lịch sử hoạt động
- Phục vụ báo cáo

CHƯƠNG 4: THỬ NGHIỆM VÀ ĐÁNH GIÁ

1. Kết quả thử nghiệm và đánh giá hệ thống

Kết quả thử nghiệm của hệ thống được đánh giá thông qua các tiêu chí như độ ổn định truyền dữ liệu, thời gian phản hồi, độ chính xác hiển thị dữ liệu và khả năng đồng bộ giữa các thành phần trong hệ thống (ESP32 – MQTT – API – Web). Đây là những yếu tố quan trọng phản ánh hiệu quả của việc áp dụng chuẩn đóng gói dữ liệu JSON/XML trong hệ thống nhúng.

- Độ chính xác dữ liệu: ~98–100%

Dữ liệu nhiệt độ và độ ẩm được truyền từ ESP32 lên server và hiển thị trên giao diện web gần như không bị sai lệch. Điều này cho thấy việc đóng gói dữ liệu bằng JSON giúp đảm bảo tính toàn vẹn và dễ kiểm tra trong quá trình truyền.

- Thời gian phản hồi (Response Time): ~1–2 giây

Từ lúc cảm biến gửi dữ liệu đến khi hiển thị trên web mất khoảng 1–2 giây. Đây là mức chấp nhận được đối với hệ thống IoT sử dụng giao thức MQTT kết hợp API.

- Độ ổn định hệ thống: Cao

Hệ thống duy trì kết nối liên tục giữa các thành phần. Khi mất kết nối tạm thời (WiFi hoặc MQTT), hệ thống có thể tự động kết nối lại mà không cần khởi động lại toàn bộ.

2. Đánh giá quá trình truyền và đóng gói dữ liệu

Trong hệ thống, dữ liệu được chuẩn hóa dưới dạng JSON trước khi truyền đi, ví dụ:

```
Json
{
  "temperature": 28.5,
  "humidity": 65,
  "led_temp": true,
  "led_hum": false,
  "createdAt": "2026-04-30T10:00:00Z"
}
```

Việc sử dụng JSON mang lại nhiều lợi ích rõ rệt:

- Dữ liệu có cấu trúc rõ ràng, dễ đọc và dễ debug
- Tương thích tốt với nhiều nền tảng (ESP32, Node.js, Web)
- Dễ dàng mở rộng khi thêm cảm biến hoặc chức năng mới

Bên cạnh đó, hệ thống cũng có thể hỗ trợ XML trong các trường hợp cần tích hợp với các hệ thống cũ hoặc yêu cầu chuẩn hóa nghiêm ngặt hơn, tuy nhiên JSON vẫn được ưu tiên do nhẹ và tối ưu hơn cho IoT.

3. Đánh giá giao tiếp giữa các thành phần

- ESP32 → MQTT Broker:
Dữ liệu được publish theo topic định sẵn, đảm bảo truyền nhanh và nhẹ.
- MQTT → Backend (Node.js API):
Server nhận dữ liệu và lưu trữ vào database, đồng thời cung cấp API cho frontend.
- Frontend (Web) → API:
Web sử dụng fetch API để lấy dữ liệu và hiển thị theo thời gian thực.
- MQTT Control (2 chiều):
Người dùng có thể điều khiển LED từ giao diện web, gửi lệnh xuống ESP32 thông qua MQTT.

Hệ thống hoạt động đúng theo mô hình publish/subscribe, giúp giảm tải server và tăng tính linh hoạt.

CHƯƠNG 5: KẾT LUẬN VÀ KIẾN NGHỊ

5.1 Kết luận

Hệ thống chuẩn hóa và đóng gói dữ liệu sử dụng JSON/XML trong đề án đã đạt được nhiều kết quả tích cực trong quá trình thiết kế, triển khai và đánh giá thực tế. Hệ thống được xây dựng dựa trên mô hình IoT gồm các thành phần chính: vi điều khiển ESP32, giao thức MQTT, server trung gian (Node.js API) và giao diện web hiển thị dữ liệu.

Việc áp dụng chuẩn đóng gói dữ liệu JSON giúp hệ thống có thể truyền và xử lý thông tin một cách rõ ràng, đồng nhất và hiệu quả giữa các thành phần khác nhau. Đồng thời, khả năng mở rộng sang XML giúp hệ thống có tính tương thích cao với nhiều nền tảng và hệ thống khác trong thực tế.

Kết quả thử nghiệm cho thấy hệ thống hoạt động ổn định, đáp ứng tốt yêu cầu đề ra trong việc thu thập, truyền tải và hiển thị dữ liệu cảm biến theo thời gian thực.

Các kết quả chính:

- Độ chính xác dữ liệu (Data Integrity): ~98–100%

Dữ liệu nhiệt độ và độ ẩm được truyền từ ESP32 đến server và hiển thị trên web gần như không bị sai lệch, chứng tỏ việc đóng gói dữ liệu bằng JSON đảm bảo tính toàn vẹn cao.

- Thời gian phản hồi (Response Time): ~1–2 giây

Hệ thống có độ trễ thấp, đảm bảo dữ liệu được cập nhật gần như theo thời gian thực, phù hợp với các ứng dụng giám sát IoT.

- Độ ổn định hệ thống: Cao

Hệ thống duy trì kết nối liên tục giữa ESP32, MQTT Broker và server. Trong trường hợp mất kết nối tạm thời, hệ thống có khả năng tự động khôi phục.

- Hiệu quả giao tiếp dữ liệu:

Việc sử dụng JSON giúp đơn giản hóa quá trình trao đổi dữ liệu giữa:

- ESP32 → MQTT Broker
- MQTT → Server (API)
- Server → Web Client

- Khả năng điều khiển hai chiều:

Người dùng có thể gửi lệnh điều khiển từ giao diện web xuống thiết bị thông qua MQTT, đảm bảo tính tương tác thời gian thực.

5.2 Kiến nghị

Mặc dù hệ thống đã đạt được kết quả tốt, vẫn còn một số điểm có thể cải thiện để nâng cao hiệu suất và khả năng ứng dụng trong thực tế.

1. Tăng cường bảo mật hệ thống

Hiện tại hệ thống chưa áp dụng các cơ chế bảo mật nâng cao. Trong tương lai cần:

- Sử dụng HTTPS cho API
- Áp dụng xác thực người dùng (Authentication)
- Bảo mật MQTT bằng username/password hoặc TLS

Điều này giúp đảm bảo an toàn dữ liệu và tránh truy cập trái phép.

2. Tối ưu hóa tốc độ và độ trễ

Mặc dù hệ thống đã có độ trễ thấp, vẫn có thể cải thiện thêm:

- Tối ưu code phía ESP32 và server
 - Giảm kích thước payload JSON
 - Sử dụng cơ chế cache hoặc WebSocket thay cho polling
- Giúp hệ thống phản hồi nhanh hơn trong thời gian thực.

3. Mở rộng chuẩn dữ liệu XML

Hiện tại hệ thống chủ yếu sử dụng JSON. Trong tương lai có thể:

- Triển khai song song JSON và XML
- So sánh hiệu năng giữa hai chuẩn
- Tích hợp với các hệ thống yêu cầu XML

Điều này giúp tăng tính linh hoạt và khả năng tích hợp hệ thống.

4. Mở rộng số lượng thiết bị và cảm biến

Hệ thống hiện tại còn đơn giản. Có thể mở rộng thêm:

- Cảm biến ánh sáng, khí gas, bụi mịn
- Điều khiển thêm quạt, máy bơm, điều hòa
- Kết nối nhiều node ESP32

Giúp hệ thống trở thành một mạng IoT hoàn chỉnh.

5. Phát triển giao diện người dùng

Trong tương lai có thể nâng cấp:

- Ứng dụng mobile (Android/iOS)
- Dashboard chuyên nghiệp (biểu đồ nâng cao, thống kê)
- Cảnh báo theo thời gian thực (notification)

Giúp người dùng dễ dàng theo dõi và điều khiển hệ thống.

6. Tích hợp Cloud và hệ sinh thái IoT

Hệ thống có thể được mở rộng bằng cách:

- Kết nối với nền tảng Cloud (Firebase, AWS, ThingsBoard)
- Lưu trữ dữ liệu dài hạn
- Phân tích dữ liệu thông minh

TÀI LIỆU THAM KHẢO

- [1] MQTT.org, *MQTT Version 3.1.1 Protocol Specification*, 2014.
Truy cập tại: <https://mqtt.org/>
- [2] EMQX, *MQTT Broker Documentation*, 2024.
Truy cập tại: <https://www.emqx.io/docs/>
- [3] Node.js Foundation, *Node.js Documentation*, 2024.
Truy cập tại: <https://nodejs.org/en/docs/>
- [4] Express.js, *Express Web Framework Documentation*, 2024.
Truy cập tại: <https://expressjs.com/>
- [5] Arduino, *Arduino Official Documentation*, 2024.
Truy cập tại: <https://www.arduino.cc/reference/en/>
- [6] Espressif Systems, *ESP32 Technical Reference Manual*, 2023.
Truy cập tại: <https://www.espressif.com/>
- [7] Douglas Crockford, *The JSON Data Interchange Format*, RFC 8259, 2017.
Truy cập tại: <https://www.rfc-editor.org/rfc/rfc8259>
- [8] W3C, *Extensible Markup Language (XML) 1.0 Specification*, 2008.
Truy cập tại: <https://www.w3.org/TR/xml/>
- [9] Mozilla Developer Network (MDN), *JSON Guide*, 2024.
Truy cập tại: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON
- [10] Mozilla Developer Network (MDN), *Fetch API*, 2024.
Truy cập tại: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [11] HiveMQ, *MQTT Essentials Series*, 2024.
Truy cập tại: <https://www.hivemq.com/mqtt-essentials/>
- [12] Tailwind CSS, *Tailwind CSS Documentation*, 2024.
Truy cập tại: <https://tailwindcss.com/docs>
- [13] Chart.js, *Chart.js Documentation*, 2024.
Truy cập tại: <https://www.chartjs.org/docs/>